IMD3002 – 3D COMPUTER GRAPHICS

Prof. Chris Joslin

L17 – Textures I

Overview

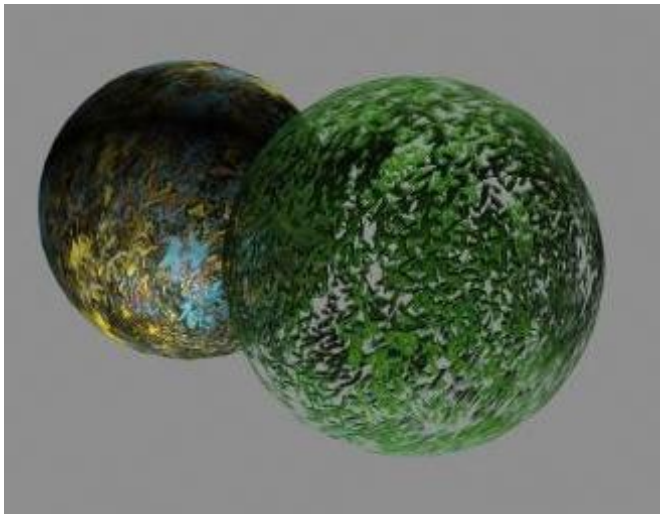
- In this lecture we take a look at texture mapping and discuss the many ways in which texture mapping can permit us to save rendering more complex objects

Texture Mapping

- Introduction
  - We cannot discuss 3D graphics without talking about textures and texture mapping – they are an essential part of the realism that is graphics today
  - In place of detailed surfaces used to simulate the varying shades of an object, we can use a simple 2D texture map to supply colour values to the renderer
  - Technically speaking, texture mapping is the addition of a 2D texture onto a 3D surface (implicit or explicit)

Texture Mapping

- Introduction
  - The application of a standard texture does not affect the smoothness of the surface, it merely “influences” (from directly replacing, to strengthening an existing material colour) the colours



Texture Mapping

- Introduction
  - **Example:** this is the classic "ShutterBug" scene from Pixar, with textures

Texture Mapping

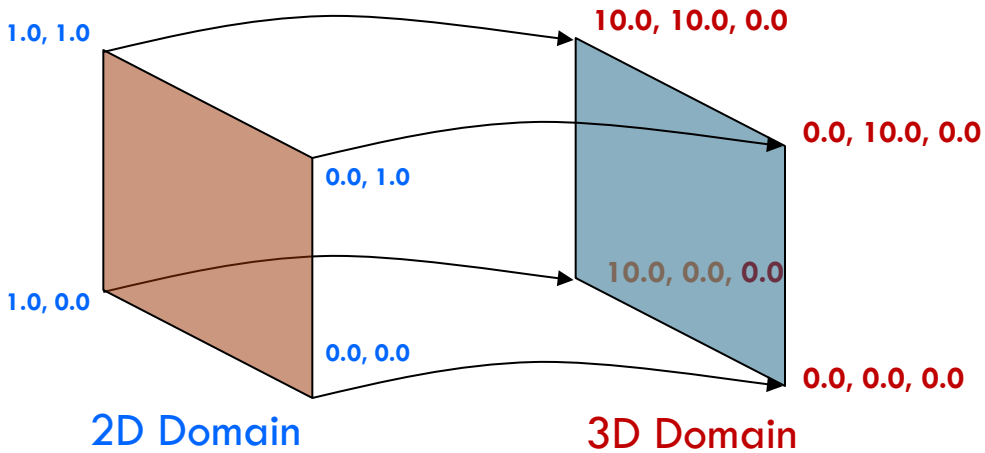
- Introduction
  - Texture Mapping involves making a correspondence between one domain (the texture) and another (the 3D scene)
  - All methods require knowledge of the 3D coordinate space (i.e. the x, y, z values on the surface)
  - In Ray Tracing, this is the surface/ray intersection point
  - In scan-line graphics it is more complicated since the objects are converted to canonical view-volume and then scan-converted

Texture Mapping

- Introduction
  - We can texture map in one of 3 domains: 1D, 2D, or 3D
  - We can use 1D textures to represent colour ramps, gradients, or any variances over one dimensions
  - We shall not address 1D in this lecture as it is not relevant to most of what we are doing and concentrate on the 2D and 3D texture mapping methods

Texture Mapping

- 2D Texture Mapping
  - Probably the easiest form of texture mapping is performing a straight 2D texture mapping
  - Here we are effectively mapping the coordinates of the texture from the 2D domain into the coordinates of the 3D domain, which is where our model exists



Texture Mapping

- 2D Texture Mapping
  - In order to map a texture from the 2D domain, into the 3D domain, we must make certain of several key factors
    - Firstly, we need to make sure that our texture is correctly placed, each coordinate on the texture must correspond to a coordinate in 3D
    - Second, we need to make sure the Euclidian distances in the 3D space are the Euclidian distances we have in 2D (or the difference is desirable factor)
    - Thirdly, in some cases we need to make sure our texture is on the correct side of the object

Texture Mapping

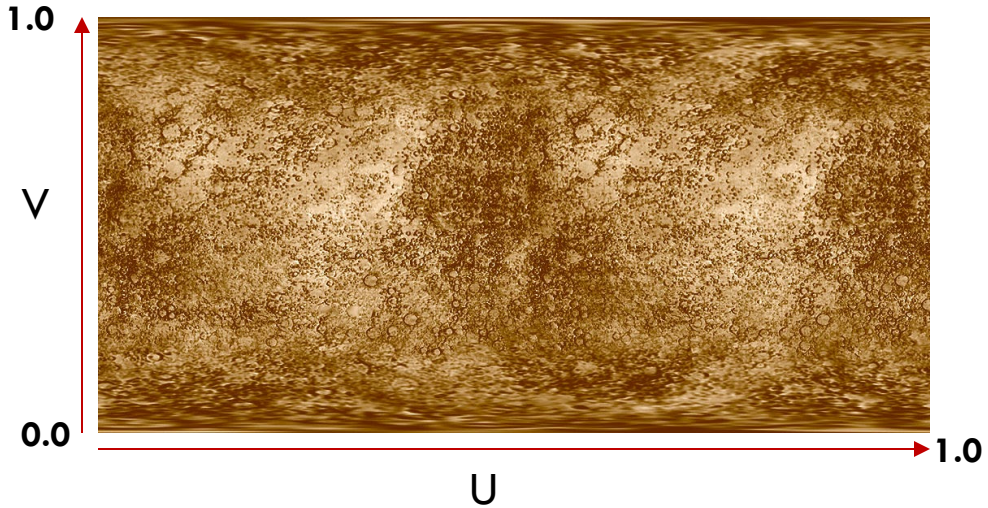
- 2D Texture Mapping
  - We can apply a texture to an object in one of two ways:
    - ***Direct Coordinates*** ~ Supplying direct texture coordinates corresponding to vertices on the object
    - ***Functions*** ~ Using a function to create a correspondence between the texture coordinates and the 3D space
  - The former is fairly simple and can be used for most objects, the second can be used for implicit objects

Texture Mapping

- 2D Texture Mapping
  - For either method we define the texture map as using standard 2D coordinates (U, V)
  - For each point in 3D space (x, y, z), we must find the corresponding point in the 2D texture map space and colour the 3D point with the value from the texture map

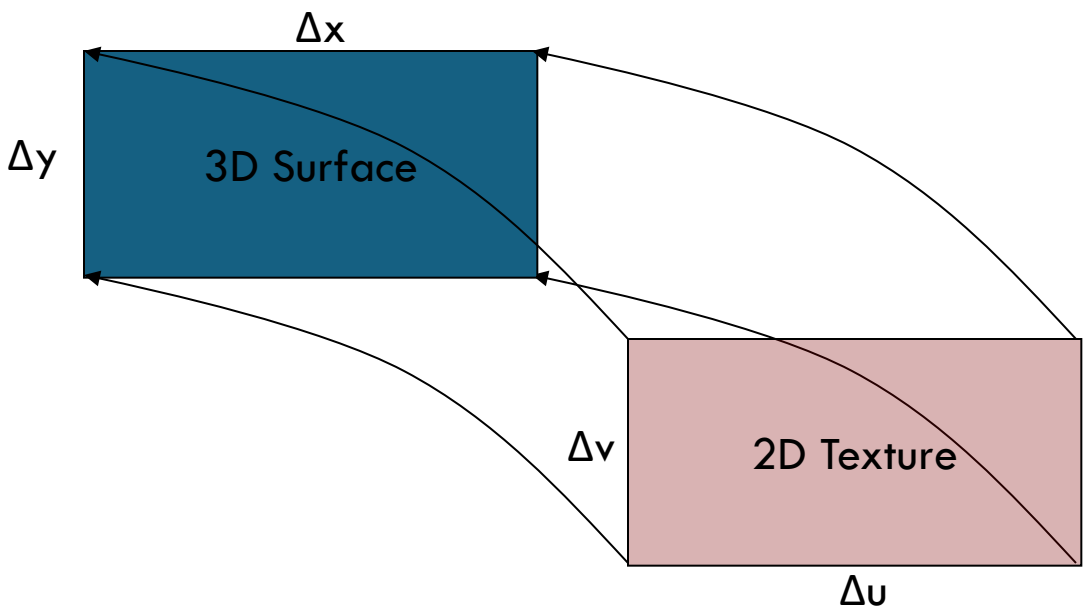
Texture Mapping

- 2D Texture Mapping : Explicit Surfaces
  - For explicit surfaces, we can define the U, V coordinates that correspond vertices X, Y, Z
  - As U and V are values between 0.0 and 1.0, representing the entire texture (regardless of actual dimensions) is always linear



Texture Mapping

- 2D Texture Mapping : Explicit Surfaces
  - Therefore, we can map directly onto a surface
  - Using the same dimensions for the 3D surface will enable us to retain the same proportions for the 2D image)



Here if:

$$\frac{\Delta u}{\Delta v} = \frac{\Delta x}{\Delta y}$$

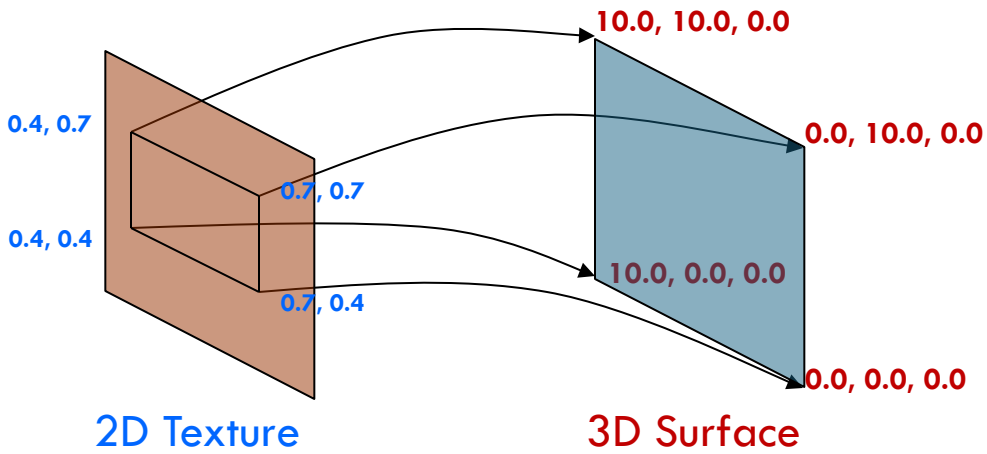
i.e. the ratios are the same, then the texture will appear correctly proportional

Texture Mapping

- 2D Texture Mapping : Explicit Surfaces
  - Therefore, if we map the coordinates of the texture not based on 1.0, 1.0 but on somewhere else in the linear scale then we can actually stretch that portion onto the same surface
  - In general, we can do this two ways:
    - ***Subsection*** ~ by specifying a value less than 1.0, we are only using a subsection of the texture
    - ***Repetition/Clamping*** ~ by specifying a value greater than 1.0 we are actually using all the texture, on less of the surface

Texture Mapping

- 2D Texture Mapping : Explicit Surfaces
  - If we use values less than 1.0 in either direction, we obtain a subpart of that original image, we can control which subpart we obtain simply by varying the U_{min} and U_{max}, and V_{min} and V_{max}



Texture Mapping

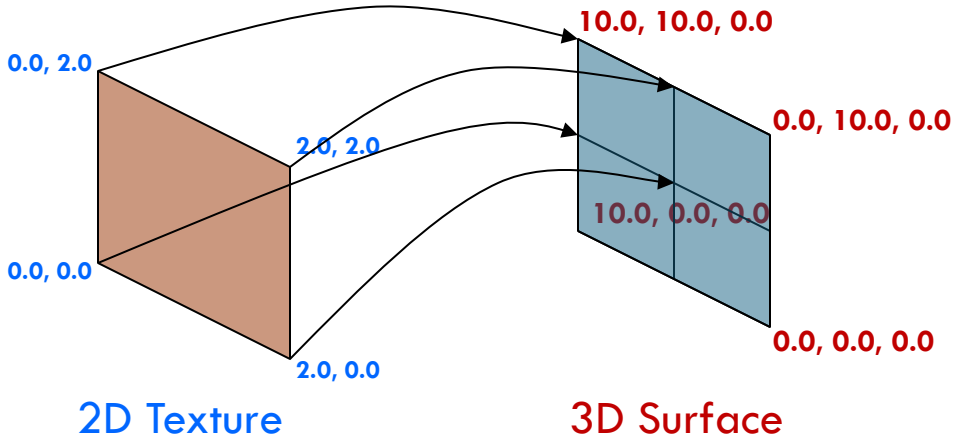
- 2D Texture Mapping : Explicit Surfaces
  - The other advantage to using subsections of a texture in this way is that we can load one texture and reuse it for various purposes
  - For example, we can load in a skybox texture (discussed later) as a single image (rather than each side)
  - We can also use a single texture for different objects, or load in textures for a person from a single image

Texture Mapping

- 2D Texture Mapping : Explicit Surfaces
  - If we use values greater than 1.0 in either direction, we are basically applying an image to a subpart of that surface
  - How that is handled depends on the texture mapping method used
  - We can either repeat, whereby the texture is applied many times in one or both directions
  - Or we can clamp, whereby the texture applied only once

Texture Mapping

- 2D Texture Mapping : Explicit Surfaces
  - The repeated texture is easy to assess, it is simply repeated the number of times based on the factor of 1.0 (being the full image)
  - **Example:** if we use a value of 2.0, then the image is repeated twice, 3.0 = three times, etc



Texture Mapping

- 2D Texture Mapping : Explicit Surfaces
  - If the texture is not repeated it is clamped, in this case the texture resides in one area (defined by the texture coordinates) results for this vary and from rendering system to rendering system
  - In general, it can have strange effects to actually clamp the texture and in many respects it is easier to setup the texture (or the surface) in order to avoid clamping

Texture Mapping

- 2D Texture Mapping : Implicit Surfaces
  - We can define our mapping with parametrically defined surfaces (such as NURBS or Bezier Surfaces)
  - In general, it requires two parameters to define a surface, i.e. we will have equations of form:

$$x = X(u,v); y = Y(u,v); z = Z(u,v)$$

Texture Mapping

- 2D Texture Mapping : Implicit Surfaces
  - We can do this with a sphere

$$X = r\sin(\pi v)\times\cos(2\pi u)$$

$$Y = r\sin(\pi v)\times\sin(2\pi u)$$

$$Z = r\cos(\pi v)$$

$$v = \arccos(Z/r)/\pi$$

$$u = (\arccos((X/r)\times\sin(\pi v)))/2\pi$$

  - Where:
    - U and V are the texture coordinates
    - r is the radius of the sphere
    - X, Y, Z are the points in 3D space

Note: you do not need to remember these equations

Texture Mapping

- 2D Texture Mapping : Implicit Surfaces
  - We can also do it for a more complex implicit object, we can simply compute X, Y, and Z using U and V and the equation for a torus

$$X = (r_i + r_o \cos(2\pi v))\cos(2\pi u)$$
$$Y = r_o \sin(2\pi v)$$
$$Z = (r_i + r_o \cos(2\pi v))\sin(2\pi u)$$

  - Where:
    - U and V are the texture coordinates
    - r_i and r_o are the inner and outer radii of the torus
    - X, Y, Z are the coordinates in 3D space

Note: you do not need to remember these equations

Skyboxes & Skydomes

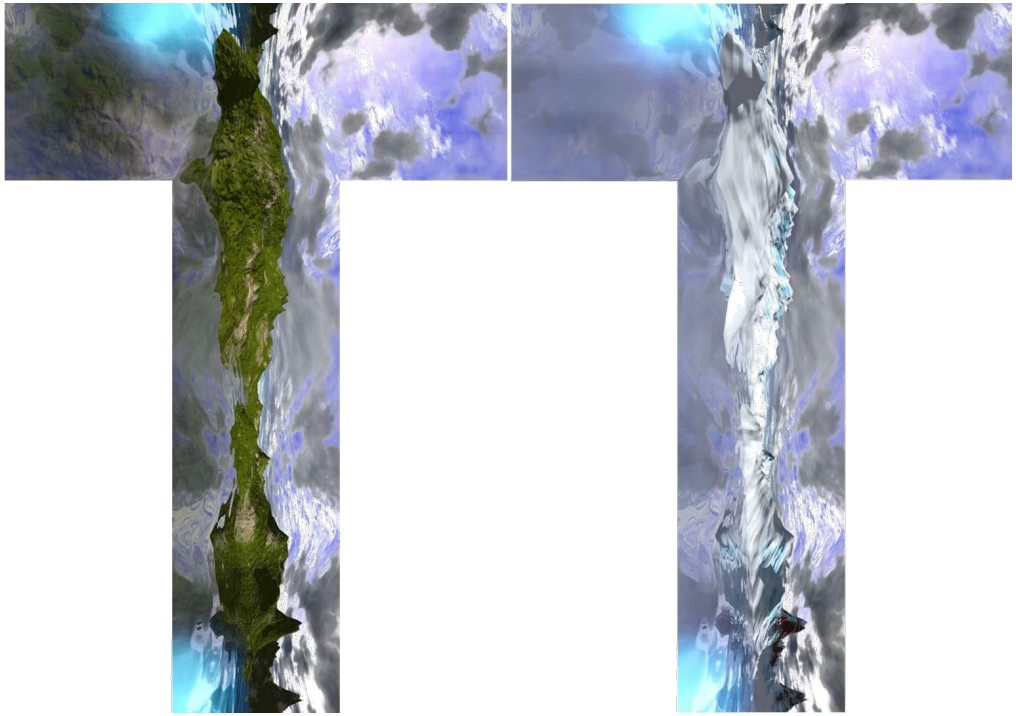
- Introduction
  - We can use a very simple texturing layout in order to create a background for any environment (normally one use for real-time) that can make the environment feel larger than it really is
  - We do this in order to render huge skylines and <u>very</u> distant objects (objects that we would not normally be able to access)
  - This technique is very useful for games where we want to render nice environments without needing masses of additional polygons

Skyboxes & Skydomes

- Introduction
  - We do this by placing a surface around the entire environment, then placing a texture map onto this surface
  - In order to create this fake environments, we employ one of two methods:
    - ***Skyboxes*** ~ constructed by mapping a texture onto a cube shape
    - ***Skydomes*** ~ constructed by mapping a texture onto a spherical, or hemispherical object
  - As long as textures are aligned properly, a viewer in the centre of the environment will have the illusion of there being an actual environment

Skyboxes & Skydomes

- Introduction
  - **Example:** here are some examples of skybox maps laid out ready for mapping onto the inside of a cube



Skyboxes & Skydomes

- Introduction
  - **Example:** here is an example of a skybox used in the Cry Engine

Skyboxes & Skydomes

- How it Works
  - As a viewer moves through a 3D scene, it is common for the skybox to remain stationary with respect to the viewer
  - This technique gives us the illusion of the environment being very far away since other objects in the scene appear to move, whilst the skybox does not
  - This imitates the stereo effect we are used to: where distant objects (e.g. clouds, stars and mountains, etc) appear stationary when the viewpoint is changed by small distances

Skyboxes & Skydomes

- How it Works
  - Everything in a skybox will always appear to be infinitely distant from the viewer
  - It is therefore necessary to be careful when designing skyboxes in order not to include images of discrete objects that will give the viewer a false perspective of the image as the scene is traversed
  - The source of a skybox can be any form of texture including photographs, drawings, or pre-rendered 3D geometry

Mipmapping

- Introduction
  - Objects in a scene are rendered at different distances (near and far) from the viewing camera, any objects with textures applied will also be handled in the same way
  - As a textured object moves closer or further away from the camera, the textures must be filtered in order to be applied to the more distant objects
  - As the filtering is based on distance - flickering occurs as the camera moves, this is due to the decimation process

Mipmapping

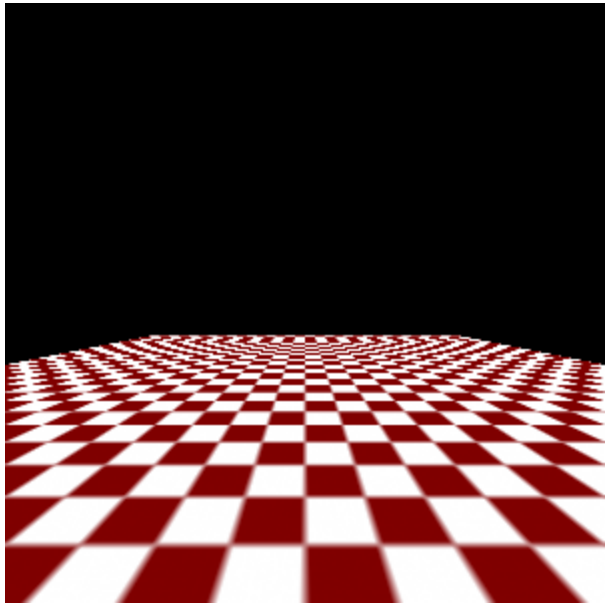
- Introduction
  - The solution to this problem is ***mipmapping***
  - Mipmapping is a process where a set of filtered texture maps of decreasing resolution are generated from a single high resolution texture prior to use
  - We use this to reduce flickering during texture mapping and display
  - Each individual mipmap is a distinct texture based on the original; it is a downsized and filtered version of the previous mipmap level

Mipmapping

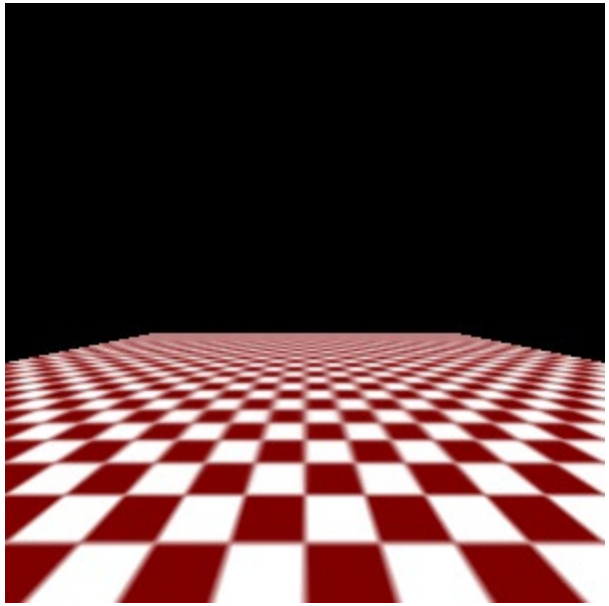
- Introduction
  - Mipmapping allows the rendering system to apply the appropriate level of detail for a texture when rendering rather than shrinking the original texture
  - We can apply different mipmaps to combine into a single texture; i.e. we are not constrained to using only one mipmap for a specific surface

Mipmapping

- Introduction
  - **Example:** a tiled floor based on a texture; the image on the left is not mipmapped, whereas the image on the right is



Simple Textured Surface



Mipmapped Surface

Mipmapping

- Generating Mipmaps
  - Each mipmap level is generated by taking half the size of the previous mipmap and then applying a filter to it
  - The filtering method depends on the system, but is basically a decimation process, and as the values are factors of 2, it can be done quite easily
  - If we have a base image that is 64x64 in size, the lower levels of detail will be: 32x32, 16x16, 8x8, 4x4, 2x2 and 1x1

Advanced Mapping

- Texture maps are interesting in themselves, they add more realism and detail, however they have more uses and can add more detail when we apply them in different ways
- These advanced mapping techniques include:
  - Bump Mapping
  - Normal Mapping
  - Displacement Mapping
  - Light/Illumination Mapping
  - Environmental/Reflection Mapping

Bump Mapping

- Introduction
  - Bump Mapping is a technique that is used to make a surface appear perturbed in some manner; bumpy, wrinkled, wavy, etc
  - This results in is a more detailed surface representation that more closely resembles the details inherent in our natural world
  - We use bump mapping to effectively simulate a surface that would utilise many polygons – which is obviously inefficient for our rendering engine (i.e. more polygons = slower rendering speed)

Bump Mapping

- Introduction
  - In place of using a texture to change the colour component of a surface, we use it instead to perturb the surface normal
  - The geometric normal (i.e. the perpendicular vector to the surface) remains the same, we merely modify the normal used in our lighting model (e.g. Phong)
  - If we use the normal per vertex, we will have a smooth surface between polygons, but if we use normal per pixel, this is where we end up with the bumpy surface

Bump Mapping

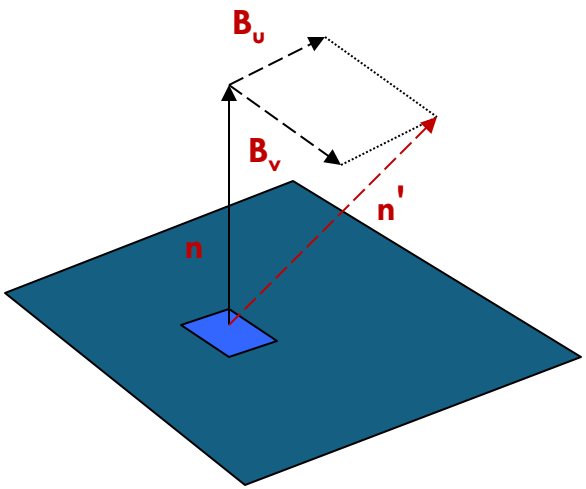
- Introduction
  - **Example:** here is a scene from Half-Life 2 where the walls are bump mapped, the actual polygons used to represent the wall are only 1 or 2 in density – the bump map does the rest

Bump Mapping

- Implementation
  - There are several ways to implement bump mapping (although the techniques are fairly similar); the two key methods are B_{UV} mapping, and height fields
  - In general though both methods work by calculating the pixel shade by comparing the vector direction point in the map
  - If the point is more at the light, then it's brighter, the points facing further away from the light actually become darker more quickly

Bump Mapping

- Implementation: B_{UV} Mapping
  - In B_{UV} mapping, we store the perturbation of the surface normal vector (n) for each pixel as two vector displacements B_U and B_V
  - This provides us with n' which is unnormalised

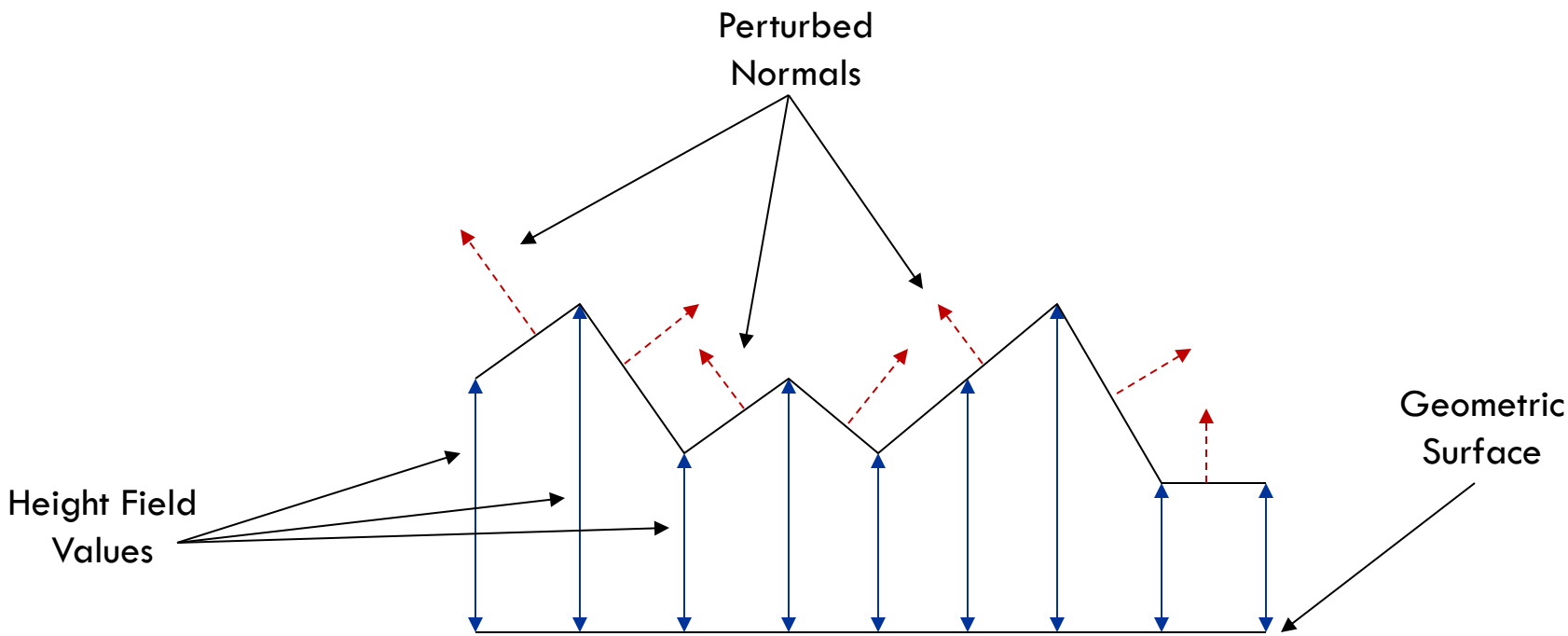


Bump Mapping

- Implementation: Height Fields
  - Using Height Fields (or Height Maps as they are sometimes referred) we use single monochrome values to represent the height – therefore a pixel is white in a high area and black in a low area
  - We can actually use this technique to create bump map, or when first scanning a bump map as we end up using the height field to generate the B_U and B_V values we have seen previously

Bump Mapping

- Implementation: Height Fields
  - Here we can see the effect the height field has on perturbing the surface normals for each pixel

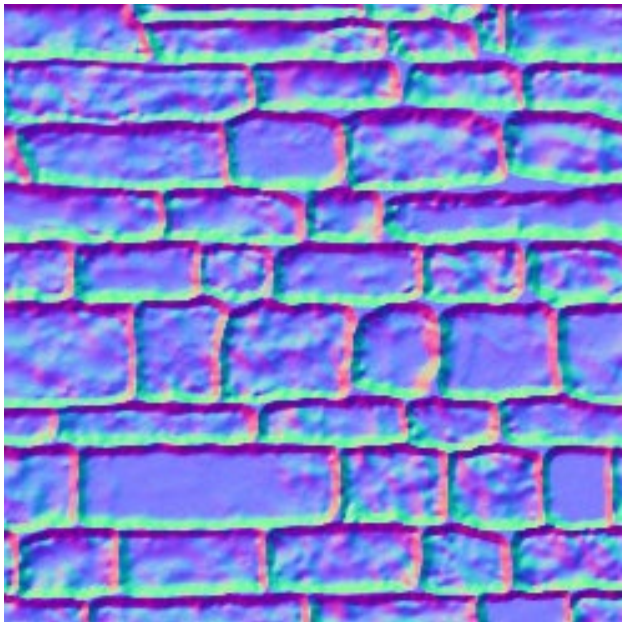


Normal Mapping

- Introduction
  - A relatively simple version of Bump Mapping is called ***Normal Mapping***, or sometimes "***Dot3 Bump Mapping***"
  - In place of storing heights or vector displacements, a normal map stores the actual normal vectors as an X, Y, Z value in the map
  - So, whilst bump mapping perturbs the existing normal of a surface, normal mapping replaces the normal entirely
  - This has advantages in terms of speed

Normal Mapping

- Introduction
  - **Example:** here is a very simple example of a wall using a normal map in order to create that brick effect



Normal Mapping

- How it Works
  - Normal Maps simply store the normal vector as the 3 components each with an 8-bit value
  - As a normal vector can be represented in each dimension using -1 → +1 this in turn is translated into a value from 0 to 255 (meaning that -1 = 0 and +1 = 255
  - **Example:** therefore a normal vector of [-1.0, 0.0, 1.0] would be stored in 24-bits, as [0, 127, 255]
  - **Note:** we can see on the previous page the colour values (RGB) representing this

Normal Mapping

- Advantages/Disadvantages
  - The main drawback to a normal map is that unlike bump maps, which can be easily created by hand, normal maps usually have to be generated
  - This usually means first creating detailed geometry and then obtaining a normal map from that
  - ATI, as well as many others, has a normal map generation tool from geometry
    - http://ati.amd.com/developer/tools.html